CHARUSAT®
CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING
DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH

DEPARTMENT OF INFORMATION TECHNOLOGY

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms

DEPSTAR

Practical 5: Doubly Linked List and Circular Linked List Operations

Practical Aim:

To implement core doubly linked list and circular linked list operations and understand how bidirectional and circular linking change the way nodes are inserted, deleted, and traversed.

Problem 1 :

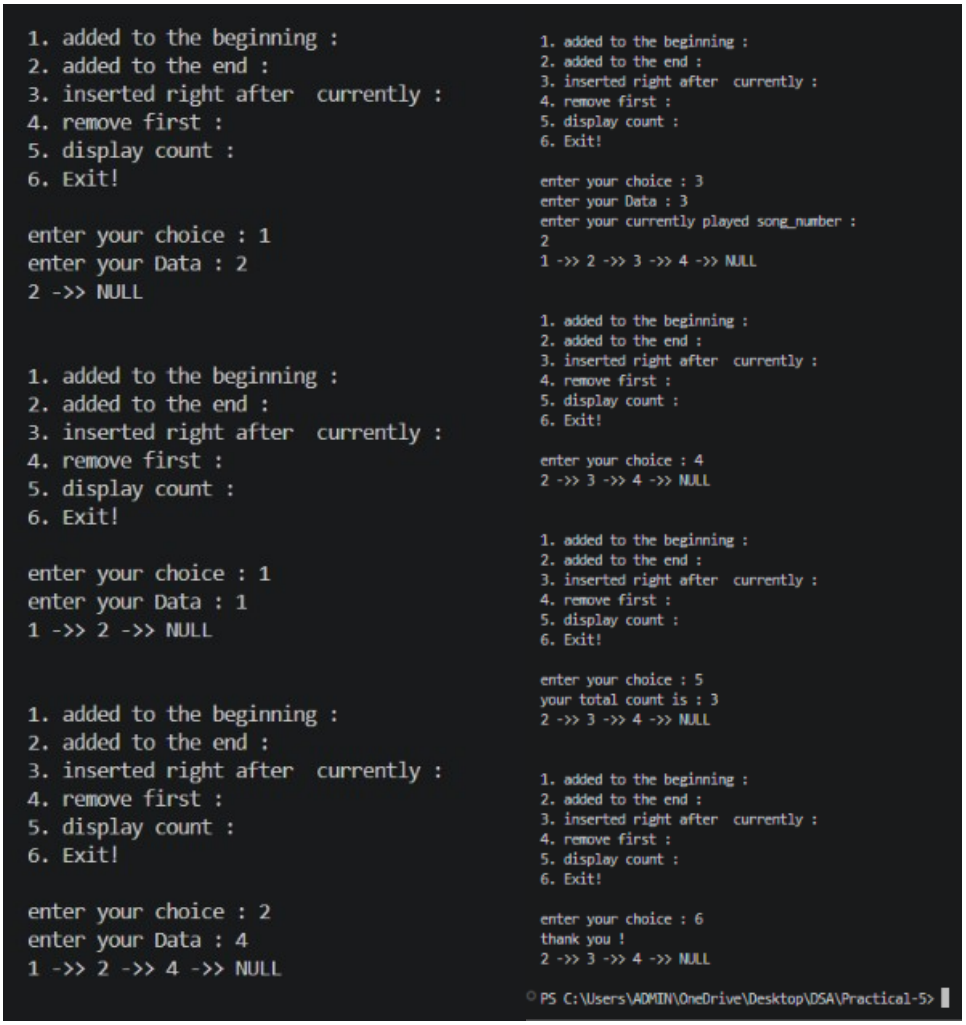
A music player maintains a playlist where songs can be added to the beginning, added to the end, or inserted right after a specific song that is currently playing. When a song is removed, only the first song in the playlist is dropped. At any point the player can count how many songs are in the playlist and display them from first to last. Given a sequence of such operations, implement all of them on a doubly linked playlist and print the result after each operation.

Describe how your insertion after a given song works. What extra step does a doubly linked list require here that a singly linked list did not? What happens if the given song is not found in the playlist?

For Code:

https://github.com/ayushposhiya00-lab/FDSA/tree/main/DSA/Practical-5

Output:



Problem Faced:

The main challenge was managing both the next and prev links correctly in a doubly linked list. Inserting a song after a given song required updating four

links in the right order, otherwise the rest of the playlist could be lost. Removing the first song also needed care: the prev link of the new first node had to be reset, and the case where the playlist contains only one song had to be handled separately. It was also necessary to decide how the program should behave when the song after which the insertion is requested is not present in the playlist.

Outcome/Conclusion:

All playlist operations (insertion at the beginning, insertion at the end, insertion after a given song, deletion of the first song, counting the songs, and forward display) were implemented on a doubly linked list, and the playlist was printed after every operation.

For insertion after a given song, the list is first searched for that song. The new node's prev is set to the given song, the new node's next is set to the given song's old next, the old next node's prev is set to the new node (only if that node exists), and finally the given song's next is set to the new node. Compared with a singly linked list, the extra step is maintaining the backward links, i.e. setting prev of the new node and updating prev of the node that follows it. If the given song is not found, the playlist is left unchanged and a "Song not found" message is displayed.

Problem 2 :

A group of students are sitting in a circle playing a passing game. A token starts at the first student and is passed around the circle one student at a time. Students can join the circle at any position and leave at any time, and the circle must remain unbroken after every join or leave. Given a sequence of join, leave, and display operations, implement them on both a singly circular and a doubly circular linked list and print the current circle after each operation.

Describe how your solution ensures the circle stays unbroken after a student leaves. What is the one case where breaking the circle is most likely to happen as a bug, and how did you handle it?

For Code:

https://github.com/ayushposhiya00-lab/FDSA/tree/main/DSA/Practical-5

Output:



Problem Faced:

Keeping the circle unbroken after every join and leave was the main difficulty. A circular list has no NULL to mark the end, so the last node must always point back to the first node, and this had to be maintained in both the singly circular and the doubly circular versions. Handling the removal of the first node, the last node, and the only node in the circle was tricky, and the traversal had to be written so that it stops after exactly one full round instead of looping forever.

Outcome/Conclusion:

Join (at any position), leave, and display operations were implemented on both the singly circular and the doubly circular linked lists, and the circle was displayed after each operation. When a student leaves, the previous node is linked directly to the next node of the leaving student (and, in the doubly circular list, the next node's prev is linked back to the previous node), and then the removed node is freed. Display stops when the traversal pointer returns to the starting node.

The case where the circle is most likely to break is removing the first (head) node or the last node. If the head is removed without updating the last node's link and the head pointer, the last node keeps pointing to a deleted node. This was handled by explicitly updating the last node's next (and the new head's prev in the doubly circular list) along with the head pointer, and by setting head to NULL when the only student leaves.

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation:

[Problem 1] A doubly linked list maintains two links per node instead of one. When deleting the front node, explain why having a backward link on the second node creates a responsibility that did not exist in a singly linked list — and what goes wrong if you forget to handle it.

Sol: In a doubly linked list, the second node's prev pointer points to the front node. When the front node is deleted, the second node becomes the new head, but its prev still points to the deleted node unless it is explicitly set to NULL. In a singly linked list there is no backward link, so moving the head pointer forward was enough. If this step is forgotten, the list looks correct in forward traversal, but the new head holds a dangling pointer to freed memory. Any later backward traversal, or any operation that uses prev (such as inserting or deleting near the head), will read invalid memory and can cause wrong output, corrupted links, or a crash.

[Problem 2] If your traversal of the circular list does not stop at the right moment, it will loop forever. Without referencing any specific code, explain what condition must be true at the moment you stop, and why that condition would fail if you set the last node's link to null.

Sol: In a circular list the traversal must stop at the moment the moving pointer comes back to the node where the traversal started (the head), after visiting every node exactly once. That is the only signal that one full round is complete, because no node has a NULL link to indicate the end. If the last node's link were set to null, no node would ever point back to the head, so the "returned to start" condition could never become true. The traversal would either run off the end and fail on a null pointer, or, if it only checked for the head, never stop at the proper point. Setting the last link to null also destroys the circular property itself, turning the list into an ordinary singly linked list.